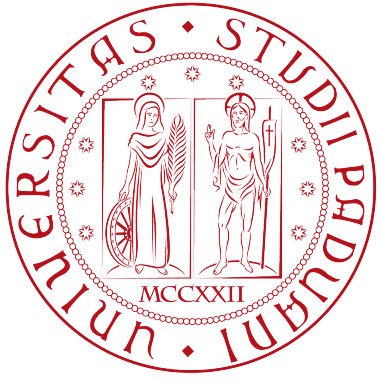
08-09-26

N-body problem

Modern Computing for Physics

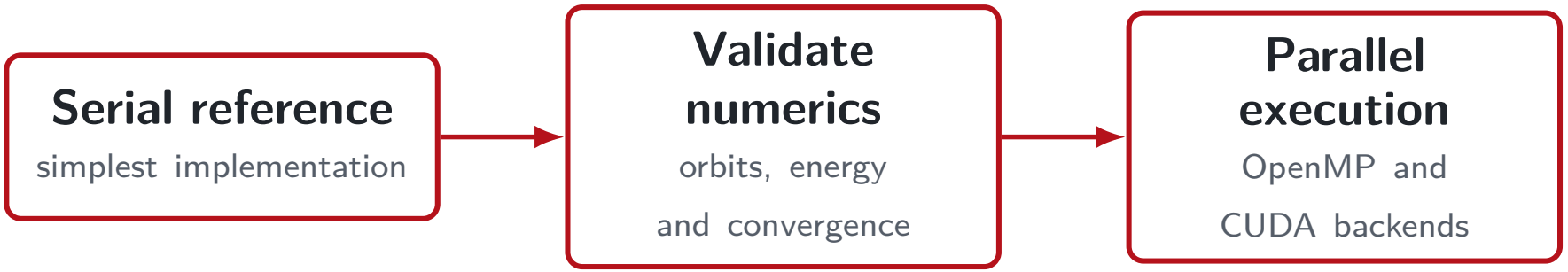
Chiara Tramarin



Start from a correct baseline

Goal

Build a direct N-body solver, validate its numerical behavior, and progressively parallelize it on CPU and GPU.



Validation rule

A parallel backend is benchmarked only after its results agree with the serial reference within the chosen numerical tolerance.

The $\mathcal{O}(N^2)$ force loop dominates cost

For each target particle i, the acceleration is obtained by summing the contributions from all other particles j.

$$\mathbf{a}_i = G \sum_{\substack{j=0 \\ j \neq i}}^{N-1} m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{\left(\|\mathbf{r}_j - \mathbf{r}_i\|^2 + \epsilon^2\right)^{3/2}}.$$

Softening

The parameter ϵ regularizes close encounters and prevents the interaction from becoming singular at zero separation.

All-pairs cost

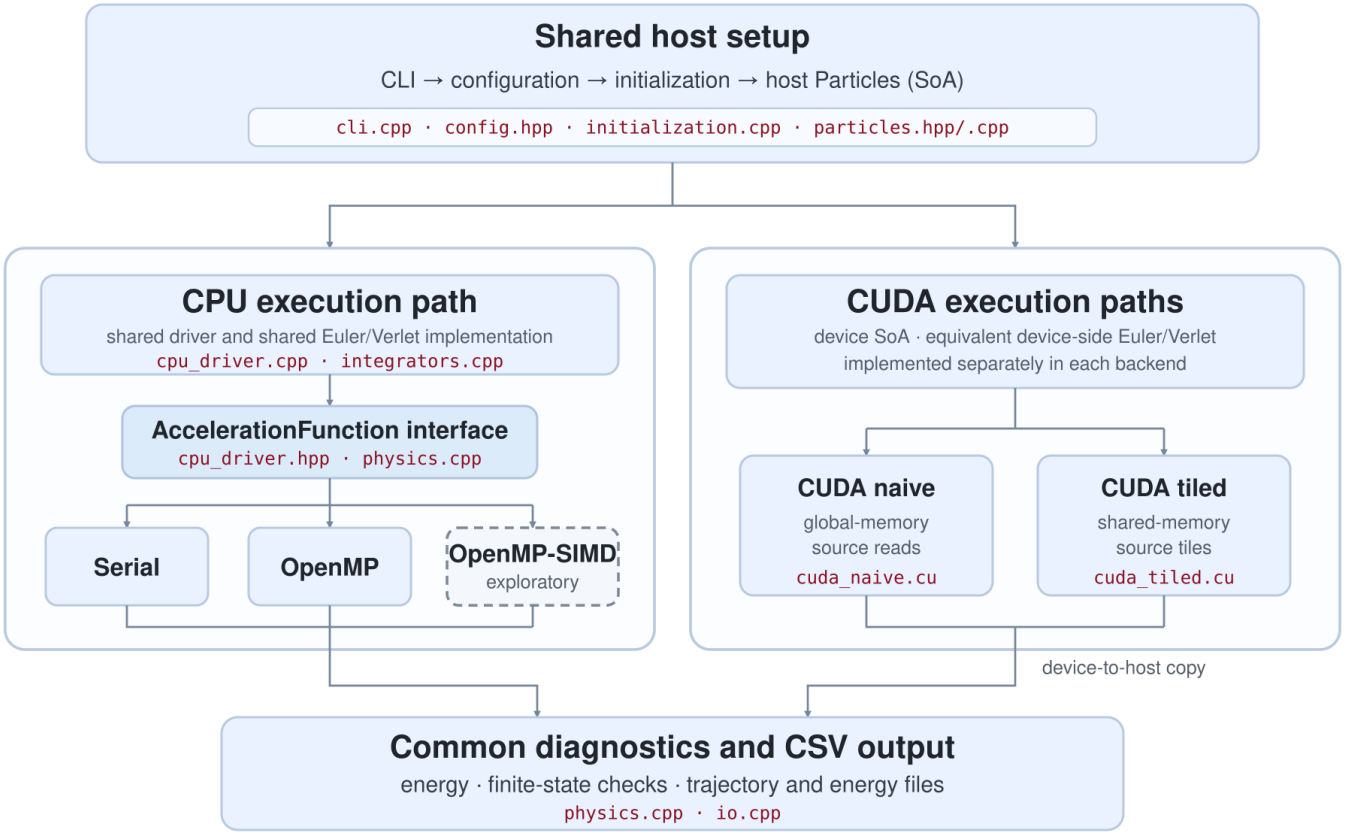
The outer loop considers N target particles, while the inner loop considers $N-1$ sources for each target:

$$N(N-1) \simeq \mathcal{O}(N^2)$$

interaction contributions per force evaluation.

The acceleration evaluation is therefore the dominant computation and the main target for parallelization.

Structure of the N-body solver



All backends receive the same initial state and produce the same diagnostics and output files, while the execution path is specialized for CPU or GPU.

Why Structure of Arrays?

One contiguous array per physical quantity

```
x    = [x0, x1, x2, ...]
y    = [y0, y1, y2, ...]
z    = [z0, z1, z2, ...]

vx   = [vx0, vx1, vx2, ...]
vy   = [vy0, vy1, vy2, ...]
vz   = [vz0, vz1, vz2, ...]

mass = [m0, m1, m2, ...]
```

During the force loop, positions and masses are read sequentially without loading complete particle objects.

CPU locality

Contiguous traversal improves spatial locality and cache use.

GPU access

Adjacent threads can read adjacent elements, enabling coalesced global-memory accesses.

Shared design

The same logical data layout is preserved across all backend implementations.

SoA matches the access pattern of the force loop: each required quantity is traversed separately and contiguously.

Correctness before performance



1

Controlled two-body references

Circular and elliptical orbits

Known orbital behaviour is used to test trajectories, orbital periods and barycentric motion.

2

Numerical behaviour

Timestep convergence and long-term conservation

Energy, center of mass and momentum are monitored while comparing Euler and Velocity Verlet.

3

General N-body test

Seeded random $N = 32$ system

Timestep and softening sensitivity are tested during collective evolution and close encounters.

- **Velocity Verlet was selected for the performance study because it showed second-order convergence and better long-term energy behaviour.**
- **All parallel implementations were validated against the serial reference before measuring performance.**

Serial reference: one target at a time

Core acceleration loop

```
for (std::size_t i = 0; i < n; ++i) {
  double axi = 0.0, ayi = 0.0, azi = 0.0;

  for (std::size_t j = 0; j < n; ++j) {
    if (i == j) continue;

    const double dx = p.x[j] - p.x[i];
    const double dy = p.y[j] - p.y[i];
    const double dz = p.z[j] - p.z[i];

    const double r2 =
        dx*dx + dy*dy + dz*dz + eps2;
    const double inv_r = 1.0 / std::sqrt(r2);
    const double inv_r3 = inv_r * inv_r * inv_r;
    const double f = cfg.G * p.mass[j] * inv_r3;

    axi += f*dx; ayi += f*dy; azi += f*dz;
  }

  p.ax[i] = axi; p.ay[i] = ayi; p.az[i] = azi;
}
```

Trade-off: using $\mathbf{F}_{ij} = -\mathbf{F}_{ji}$ would halve pair evaluations, but would require coordinated updates of particles i and j.

Directed all-pairs calculation

- one iteration computes one target acceleration a_i;
- contributions are accumulated locally;
- each iteration writes only to its own output element.

Independent writes allow the outer loop to be parallelized without race conditions.

OpenMP: parallelize independent targets



Parallelization pattern

```
#ifdef _OPENMP
#pragma omp parallel for schedule(static)
#endif
for (std::size_t i = 0; i < n; ++i) {
  double axi = 0.0;
  double ayi = 0.0;
  double azi = 0.0;

  for (std::size_t j = 0; j < n; ++j) {
    if (i == j) continue;

    // Same force calculation as in
    // the serial implementation
    axi += factor * dx;
    ayi += factor * dy;
    azi += factor * dz;
  }

  p.ax[i] = axi;
  p.ay[i] = ayi;
  p.az[i] = azi;
}
```

Target ownership

- OpenMP distributes target particles *i* among CPU threads;
- the three accumulators are private to each iteration;
- different iterations write to different array elements;
- no atomic operations or critical sections are required.

SIMD variant

An additional `omp simd` reduction was tested on the inner loop, but it did not provide a systematic performance gain.

CUDA naive: one target per GPU thread



Thread-to-particle mapping

```
const std::size_t i =
    blockIdx.x * blockDim.x + threadIdx.x;
if (i >= n) return;
const double xi = x[i];
const double yi = y[i];
const double zi = z[i];
double axi = 0.0, ayi = 0.0, azi = 0.0;
for (std::size_t j = 0; j < n; ++j) {
  if (i == j) continue;
  const double dx = x[j] - xi;
  const double dy = y[j] - yi;
  const double dz = z[j] - zi;
  const double r2 =
      dx*dx + dy*dy + dz*dz + eps2;
  const double inv_r = 1.0 / sqrt(r2);
  const double inv_r3 = inv_r * inv_r * inv_r;
  const double factor = g * mass[j] * inv_r3;
  axi += factor*dx;
  ayi += factor*dy;
  azi += factor*dz;
}
ax[i] = axi;
ay[i] = ayi;
az[i] = azi;
```

The OpenMP decomposition is preserved, but each target is now assigned to a separate GPU thread.

From OpenMP to CUDA

- one target particle per GPU thread;
- the particle state remains in device memory across time steps;
- every thread reads all source particles directly from global memory.

Same ownership rule

Each thread accumulates and writes only its own target acceleration, so no atomics are required.

CUDA tiled: reuse sources within each block

Tiling pattern

```
extern __shared__ double shared[];
const std::size_t i =
    blockIdx.x * blockDim.x + threadIdx.x;
const bool active = i < n;
for (std::size_t tile_start = 0;
     tile_start < n;
     tile_start += blockDim.x) {
  const std::size_t source =
      tile_start + threadIdx.x;
  if (source < n) {
    tile_x[threadIdx.x] = x[source];
    tile_y[threadIdx.x] = y[source];
    tile_z[threadIdx.x] = z[source];
    tile_mass[threadIdx.x] = mass[source];
  } else {
    // Zero padding for the final partial tile
  }
  __syncthreads();
  if (active) {
    for (std::size_t local_j = 0;
         local_j < tile_count; ++local_j) {
      // Accumulate from shared-memory sources
    }
  }
  __syncthreads();
}
```

Shared-memory tiling

- each block loads one source tile cooperatively;
- all target threads in the block reuse the tile.

Synchronization

Two barriers ensure that each tile is fully loaded and consumed before it is replaced.

Partial blocks

Inactive threads remain available for block-wide synchronization.

Shared-memory reuse reduces global-memory loads, but adds synchronization and extra instructions.

Benchmark setup and measurement protocol



Hardware and build

- CPU: Intel Core i7-10510U
  (4 physical cores, 8 logical threads)
- GPU: NVIDIA GeForce MX250
- Compiler: GCC 11.4, `-O3`
- CUDA target: `sm_61`

Benchmark workload

- random initial conditions;
- fixed seed $= 12345$ for an identical, reproducible workload;
- Velocity Verlet;
- 20 time steps per recorded run;
- campaign-specific particle counts, spanning $N = 128$ to $N = 32768$;
- softening $\epsilon = 0.01$.

Measurement protocol

- one unrecorded warm-up per configuration;
- seven fresh-process recorded measurements;
- execution order rotated across repetitions.

Timing scope and performance metrics

Compute time

Initial force evaluation and complete 20-step integration loop.

CPU: `std::chrono::steady_clock`

CUDA: CUDA events

Both measurements cover the initial force evaluation and the complete 20-step integration loop.

Backend time

Host wall-clock time around the complete backend call.

For CUDA, it also includes allocation, H2D transfer, event handling, synchronization and cleanup.

Diagnostic only — not used in final plots.

Performance metrics

Paired speedup

$$S_{b,r}(N) = \frac{T_{\mathrm{serial},r}(N)}{T_{b,r}(N)}$$

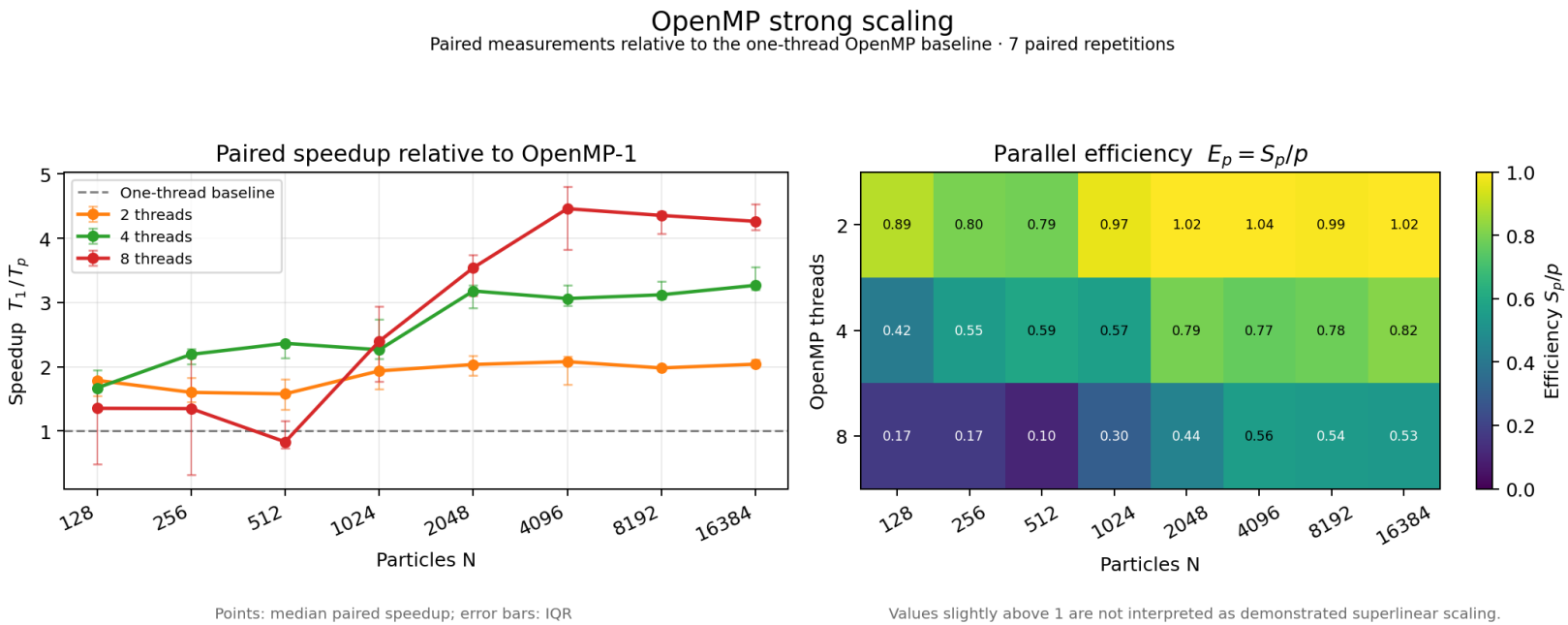
$T_{b,r}(N)$: compute time of backend b for repetition r; p: number of OpenMP threads.

OpenMP efficiency

$$E_{p,r}(N) = \frac{S_{p,r}(N)}{p}$$

For each N, ratios are computed replica by replica and summarized using the median and IQR.

OpenMP scaling with thread count

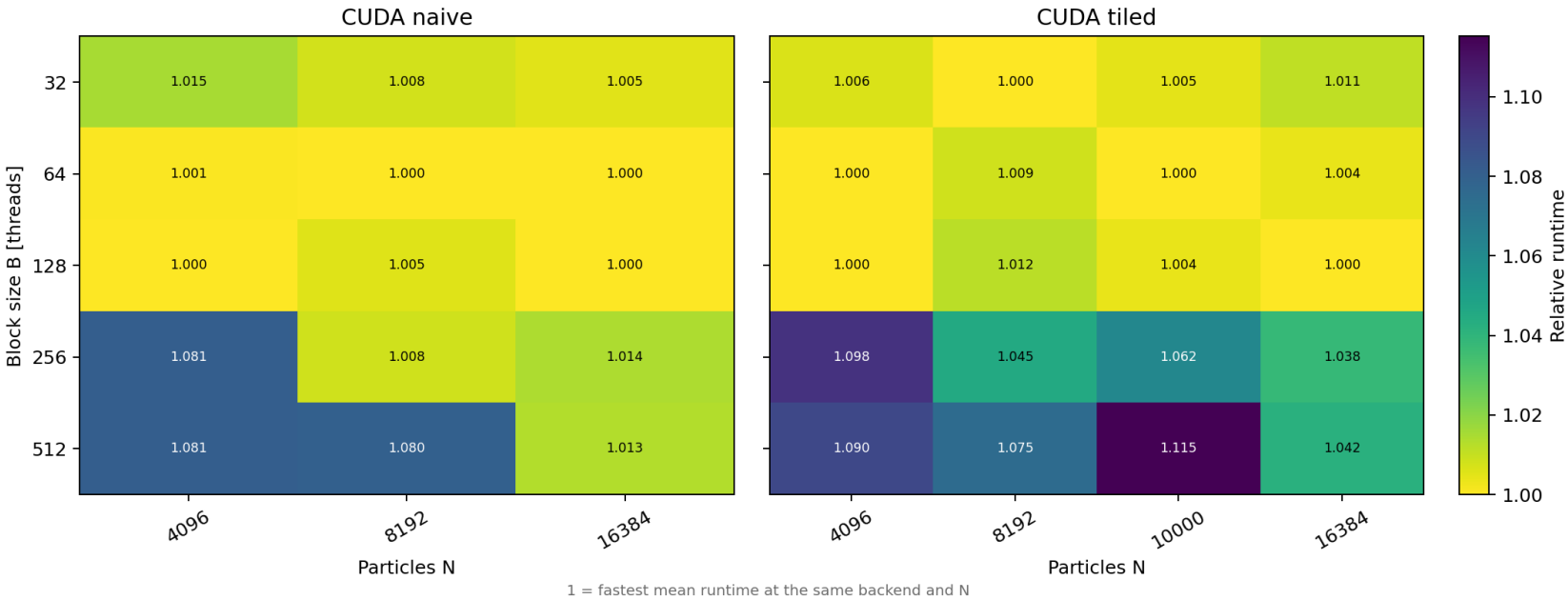


$$S_{p,r}(N) = \frac{T_{\mathrm{OpenMP}-1,r}(N)}{T_{\mathrm{OpenMP}-p,r}(N)}$$ paired speedup relative to OpenMP with one thread

Eight threads give the largest speedup at high N; four threads retain higher parallel efficiency.

OpenMP-8 was therefore retained for the final CPU comparison.

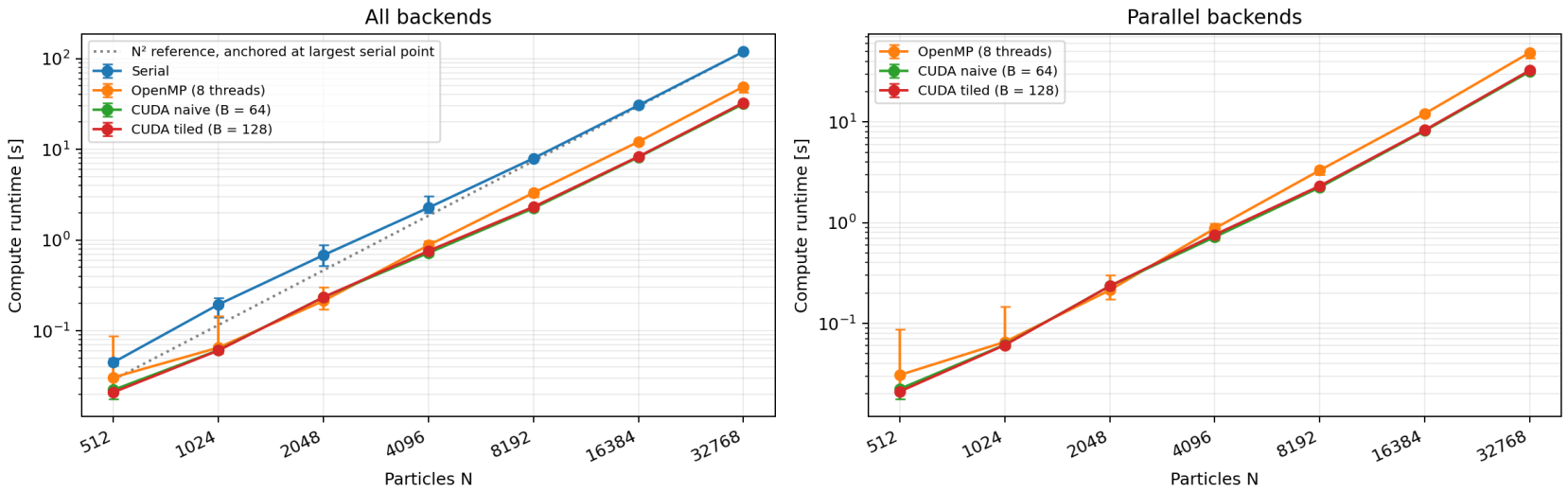
CUDA block-size tuning



Small and intermediate blocks perform similarly, while $B = 256$ and $B = 512$ are generally less competitive.

Selected configurations: CUDA naive $B = 64$; CUDA tiled $B = 128$.

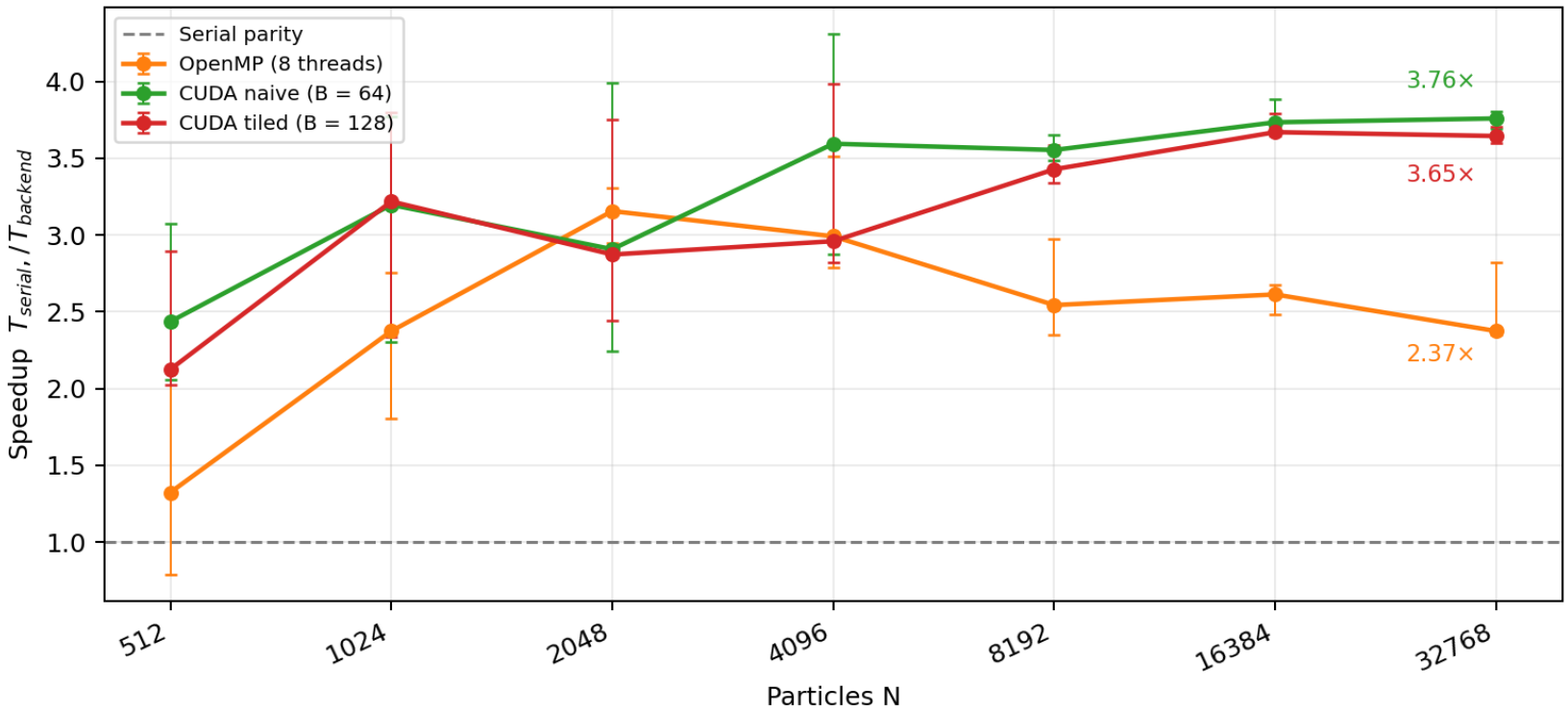
Runtime versus particle count



Points: median; error bars: IQR

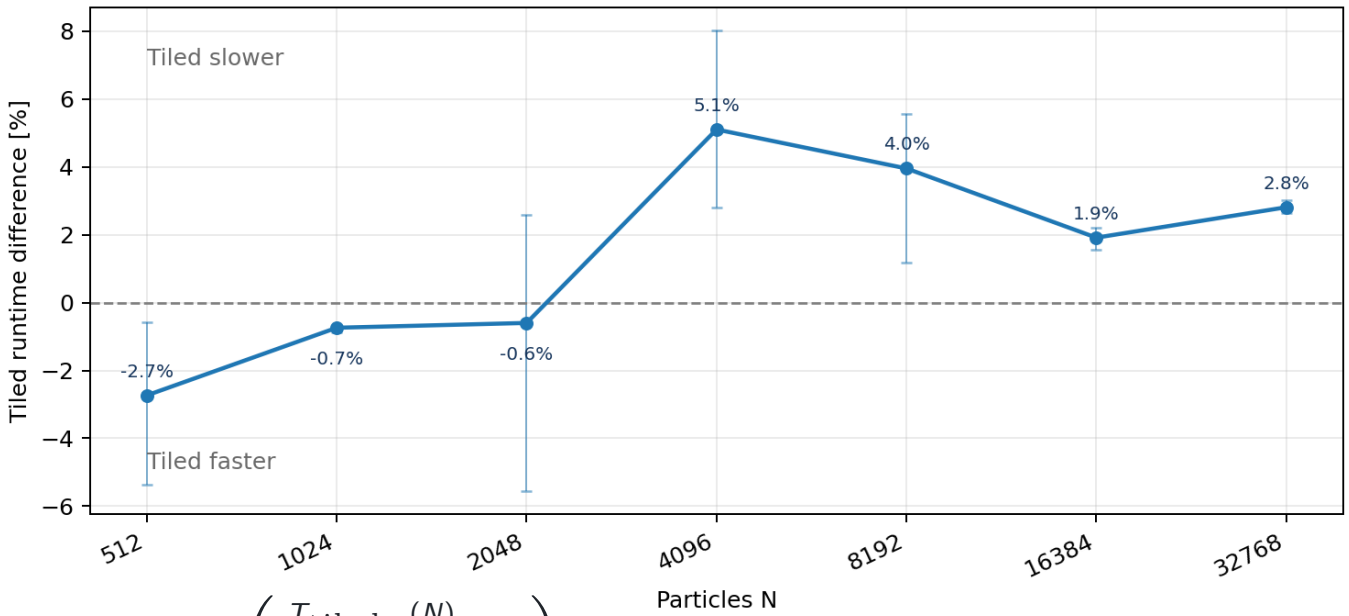
All implementations retain the expected N^2 scaling, while the tuned CUDA backends give the lowest runtime at large N.

Speedup relative to the serial implementation



At $N = 32768$, CUDA naive reaches $3.76\times$, CUDA tiled $3.65\times$, and OpenMP-8 $2.37\times$ speedup over serial.

CUDA tiled versus naive



$$\Delta_r(N) = 100 \left(\frac{T_{\mathrm{tiled},r}(N)}{T_{\mathrm{naive},r}(N)} - 1 \right)$$; negative values favour tiled, positive values favour naive.

At small N, the difference is minor and not consistently resolved; from $N = 4096$, the naive kernel is 1.9% to 5.1% faster in median.

This result motivated the profiling analysis.

Acceleration dominates GPU time

$N = 8192$, 20 Velocity Verlet steps · `nvprof` traces, PTXAS resource report and CUDA Occupancy API

Full Nsight Compute hardware-counter profiling was unavailable under WSL2.

99.96%
of profiled GPU time

is spent in the acceleration kernel

21

acceleration-kernel calls

1 initial evaluation + 20 time steps = 21

Metric	Naive B64	Tiled B128
Acceleration-kernel time	2.290 s	2.329 s
Difference from naive	—	+1.7%
Registers/thread	58	54
Dynamic shared memory/block	0 B	4096 B
Theoretical occupancy	50.00%	56.25%
Primary occupancy constraint	registers	registers

The tiled kernel reduced register use and increased theoretical occupancy, but it did not reduce acceleration time. Registers, rather than shared memory, limited theoretical occupancy in both versions.

Detailed hardware counters would be required to identify the exact cause.

Limitations and future improvements



- **Deeper GPU profiling**
  Repeat the analysis on native Linux with Nsight Compute to investigate memory behaviour and kernel stalls.
- **Single and mixed precision**
  Test whether reduced precision improves performance while preserving validated numerical accuracy.
- **Force symmetry**
  Evaluate race-free strategies for exploiting $F_{ij} = -F_{ji}$ without introducing excessive synchronization.
- **Broader hardware study**
  Repeat the benchmarks on newer GPUs and at larger N to assess the portability of the conclusions.

Thank you for your attention!

Backup slides

Two complementary validation levels

Physical validation

- known circular and elliptic motion;
- convergence under timestep refinement;
- energy, momentum and center of mass;
- sensitivity to the softening length.

Backend validation

- serial output is the numerical reference;
- identical initial state and parameters;
- component-wise final-state comparison;
- finite values and complete output structure.

Agreement between backends checks the implementation; agreement with known physics checks the model and integrator.

Euler and Velocity Verlet

Forward Euler

$$\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_n \Delta t,$$
$$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n \Delta t.$$

Global error $\mathcal{O}(\Delta t)$; used as a first-order reference.

Velocity Verlet

$$\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_n \Delta t + \tfrac{1}{2}\mathbf{a}_n \Delta t^2,$$
$$\mathbf{a}_{n+1} = \mathbf{a}(\mathbf{r}_{n+1}),$$
$$\mathbf{v}_{n+1} = \mathbf{v}_n + \tfrac{1}{2}(\mathbf{a}_n + \mathbf{a}_{n+1})\Delta t.$$

Global error $\mathcal{O}(\Delta t^2)$; time-reversible and symplectic.

Velocity Verlet was selected after the convergence and long-term energy tests.

Physical quantities monitored

Softened dynamics

$$\mathbf{a}_i = G \sum_{j \neq i} m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{(\|\mathbf{r}_j - \mathbf{r}_i\|^2 + \epsilon^2)^{3/2}},$$

$$K = \frac{1}{2} \sum_i m_i \|\mathbf{v}_i\|^2,$$

$$U = -G \sum_{i<j} \frac{m_i m_j}{\sqrt{r_{ij}^2 + \epsilon^2}}.$$

Conservation diagnostics

$$\delta_E(t) = \frac{|E(t) - E_0|}{|E_0|},$$

$$\mathbf{R}_{\mathrm{cm}} = \frac{1}{M} \sum_i m_i \mathbf{r}_i,$$

$$\mathbf{P} = \sum_i m_i \mathbf{v}_i,$$

$$\mathbf{L} = \sum_i m_i \mathbf{r}_i \times \mathbf{v}_i.$$

For an isolated system, E, $\mathbf{P}$ and $\mathbf{L}$ should remain constant, while $\mathbf{R}_{\mathrm{cm}}$ moves uniformly.

Circular two-body reference

Barycentric initial conditions

$$M = m_1 + m_2, \quad \omega^2 = \frac{GM}{(d^2 + \epsilon^2)^{3/2}},$$

$$\mathbf{r}_1 = -\frac{m_2}{M} d\,\hat{\mathbf{x}}, \quad \mathbf{r}_2 = \frac{m_1}{M} d\,\hat{\mathbf{x}},$$

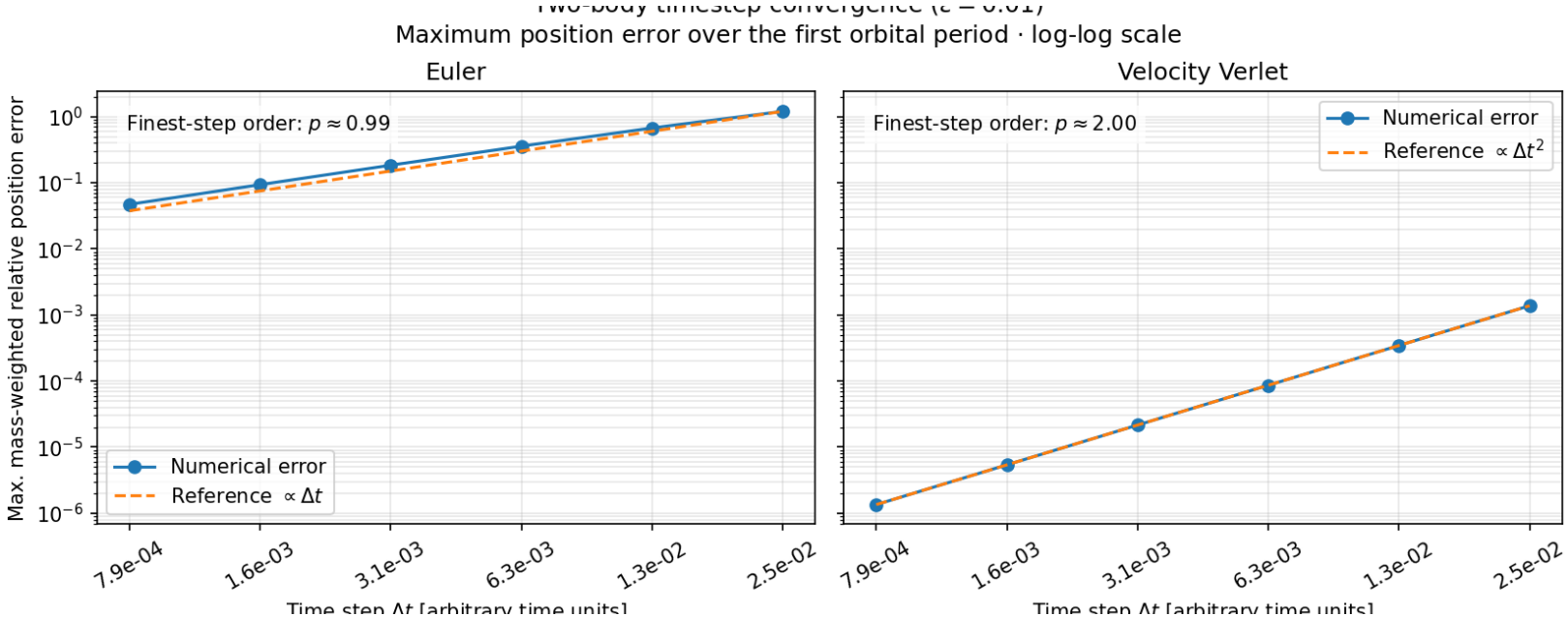
$$\mathbf{v}_k = \omega\,\hat{\mathbf{z}} \times \mathbf{r}_k, \quad T = \frac{2\pi}{\omega}.$$

Why this case is controlled

- analytical trajectory and period;
- $\mathbf{R}_{\mathrm{cm}}(0) = \mathbf{0}$;
- $\mathbf{P}(0) = \mathbf{0}$;
- constant separation in the exact orbit.

Parameters used: $m_1 = 1$, $m_2 = 10^{-3}$, $d = 1$, $G = 1$, $\epsilon = 0.01$.

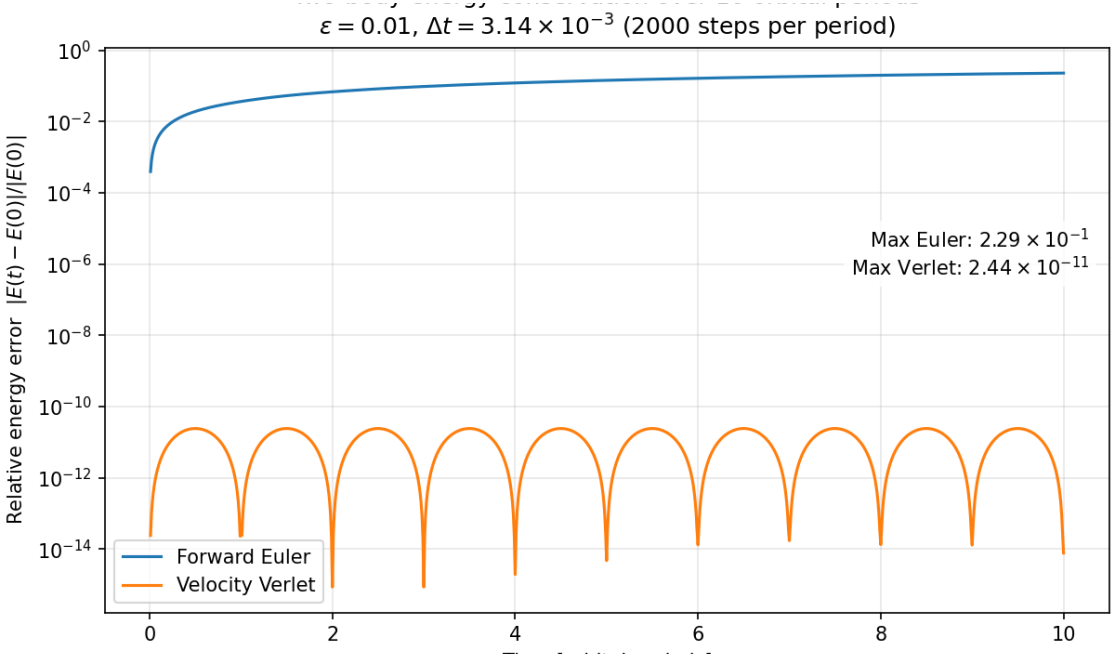
Timestep refinement confirms method order



Observed order $p = \log(e_{\Delta t}/e_{\Delta t/2})/\log 2$: Euler approaches $p = 1$, while Velocity Verlet remains at $p \simeq 2$.

This separates discretization accuracy from a single visually plausible orbit.

Long-term energy separates the integrators



Ten circular periods

Maximum δ_E:

Euler: 2.29×10^{-1}

Verlet:

2.44×10^{-11}

Verlet also gives

$\max\|\Delta \mathbf{R}_{\mathrm{cm}}\| =$

2.00×10^{-16},

$\max\|\mathbf{P}\| = 1.04 \times 10^{-17}$.

Elliptic orbit matches Keplerian geometry

Unsoftened reference: $a = 1$, $e = 0.5$, $m_1 = m_2 = 1$

$$r_p = a(1-e), \qquad v_p = \sqrt{GM\frac{1+e}{a(1-e)}},$$

$$T = 2\pi\sqrt{\frac{a^3}{GM}}, \quad r(\theta) = \frac{a(1-e^2)}{1+e\cos\theta}.$$

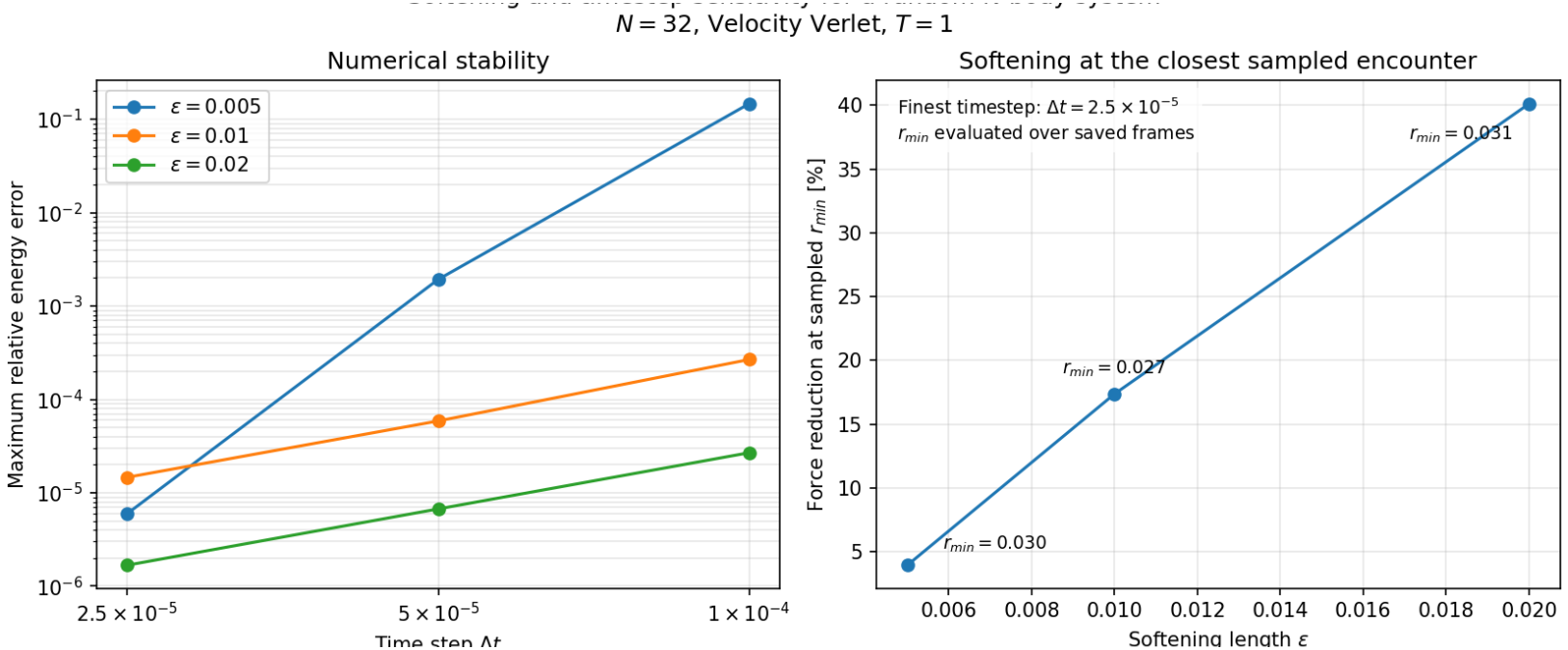
Additional checks: constant areal rate, angular momentum, center of mass and total momentum.

Measured relative errors

period	9.70×10^{-6}
semi-major axis	7.40×10^{-6}
eccentricity	7.40×10^{-6}
conic residual	2.43×10^{-5}
angular momentum	4.35×10^{-15}

All values satisfy the predefined final-profile thresholds.

Random $N = 32$ sensitivity study



Seed 12345, box size 10, minimum initial distance 0.1, one unit of simulated time.

Chosen operating point: $\epsilon = 0.01$, $\Delta t = 5 \times 10^{-5}$; the refinement study recovers approximately second-order behaviour.

Automated backend validation gate

Recorded campaign

- 46/46 backend cases marked `PASS`;
- Euler and Velocity Verlet;
- circular, elliptic and seeded random states;
- targeted partial CUDA tiles, including $N = 513$, $B = 512$.

Cross-backend state

$$|q_b - q_s| \leq 10^{-12} + 10^{-12}|q_s|$$

for every final position and velocity component.

Structural checks

- finite values;
- complete particle IDs and frames;
- constant positive masses;
- final step present;
- trajectory/energy alignment.

Negative tests also reject invalid integrators, timesteps, particle counts, block sizes and undefined relative energy errors.

Physical acceptance thresholds

Elliptic final profile

$$\max |\delta_E| < 2 \times 10^{-5},$$
$$\delta_T, \delta_a, \delta_e < 5 \times 10^{-5},$$
$$\max \delta_{\text{conic}} < 10^{-4},$$
$$\max \delta_L < 10^{-12}.$$

Circular reference

$$\max \|\Delta \mathbf{R}_{\text{cm}}\| < 10^{-12},$$
$$\max \|\Delta \mathbf{P}\| < 10^{-12},$$
$$\delta_{d,\text{final}} < 10^{-6},$$
$$\delta_{T,\text{numerical}} < 10^{-5}.$$

The energy metric is rejected when $|E_0| \leq \epsilon_{\text{mach}} \max(1, |E_0|, |E(t)|)$, because a relative error would be undefined.

Reproducible raw outputs



Trajectory CSV

`step, time, particle, x, y, z, vx, vy, vz, mass`

Validation summary

errors, observed orders, orbital diagnostics and pass/fail status

Energy CSV

`step, time, kinetic, potential, total, relative_error`

Performance campaign

raw replica CSV $\rightarrow$ summary CSV + metadata JSON $\rightarrow$ figures

Every run records the seed, integrator, timestep, softening and problem size; figures are regenerated from saved measurements.

OpenMP-SIMD was tested separately




```
#pragma omp parallel for schedule(static)
for (size_t i = 0; i < n; ++i) {
  double axi = 0.0, ayi = 0.0, azi = 0.0;
  #pragma omp simd reduction(+:axi,ayi,azi)
  for (size_t j = 0; j < n; ++j) {
    // same directed pair calculation
  }
}
```

Result

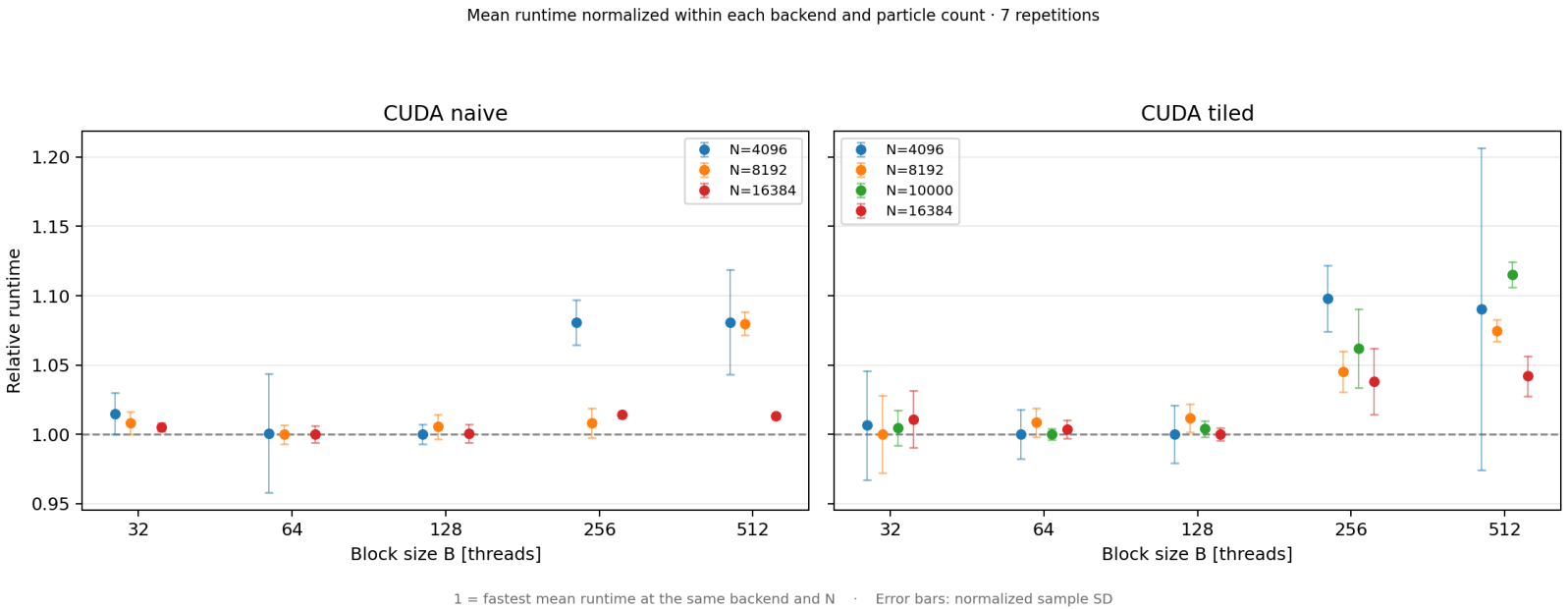
No systematic gain over standard OpenMP.

At $N = 16384$ and 8 threads,

$$\text{median } \frac{T_{\text{SIMD}}}{T_{\text{OpenMP}}} = 1.071,$$

i.e. approximately 7.1% slower.

CUDA block-size uncertainty



Values are normalized independently within each backend and N; they select robust configurations but do not compare absolute naive and tiled runtimes.

Detailed CUDA profiling summary



N=8192 · 20 Velocity Verlet steps · nvprof and PTXAS data

Metric	CUDA naive (B=64)	CUDA tiled (B=128)
Accumulated acceleration-kernel time [s]	2.290	2.329
Runtime difference relative to naive [%]	0.00	1.71
Acceleration share of profiled GPU time [%]	99.964	99.962
Acceleration-kernel calls	21	21
Position-kernel calls	20	20
Velocity-kernel calls	20	20
Host-to-device copies	7	7
Registers per acceleration thread	58	54
Static shared memory per block [B]	0	0
Dynamic shared memory per block [B]	0	4096
Theoretical occupancy	50.00%	56.25%
Limiting resource	registers	registers

$N = 8192$, 20 Velocity Verlet steps; CUDA naive $B = 64$, CUDA tiled $B = 128$.